

Learning Task-Specific Goal Costs for World Model Planning

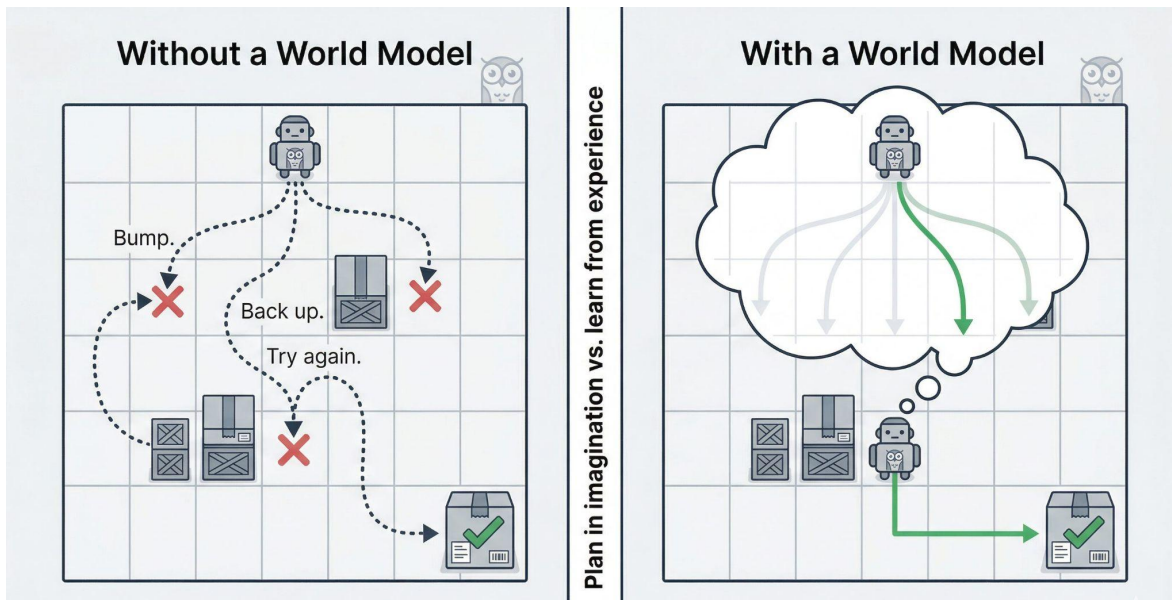
JEPA Planning Team

Amadeo De La Vega, Suchismit Ghosh,
Lukas Hakim, Junru Lin, Mukund Shankar

What is a world model

High-Level

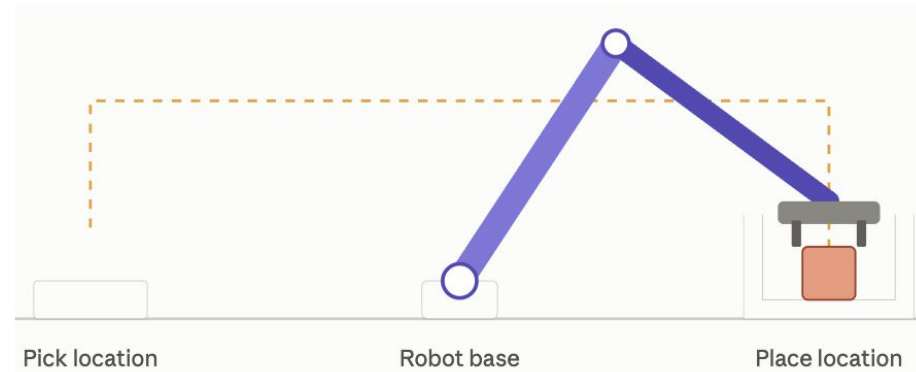
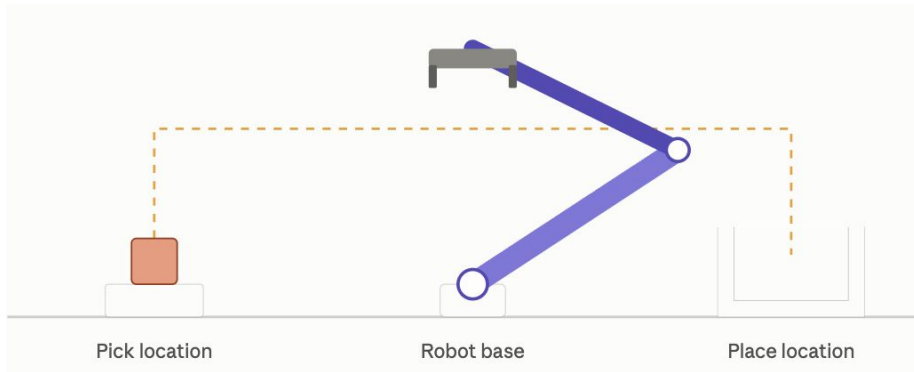
An internal model of how an environment works



What is a world model

High-Level

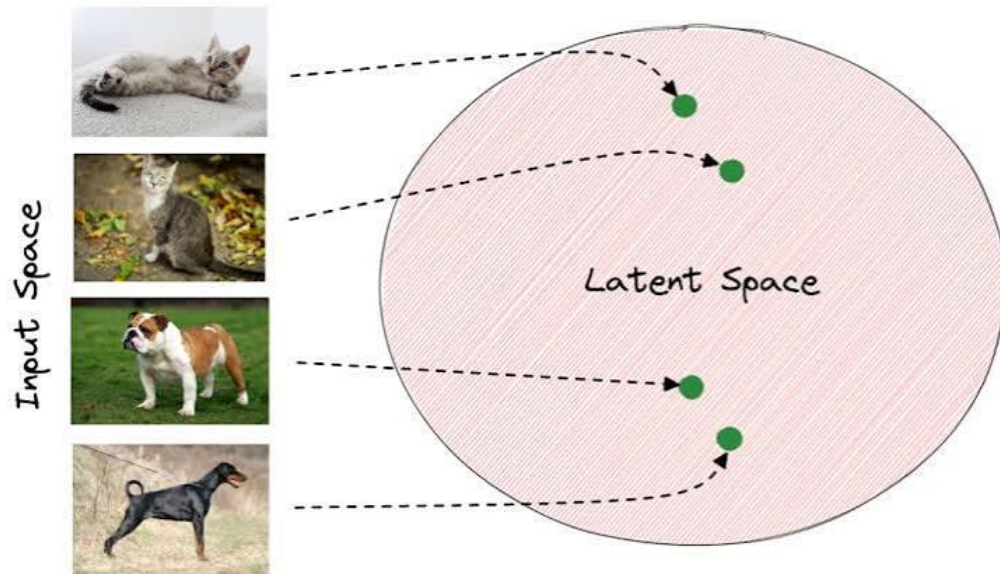
An internal model of how an environment works



What is the latent space

High-Level

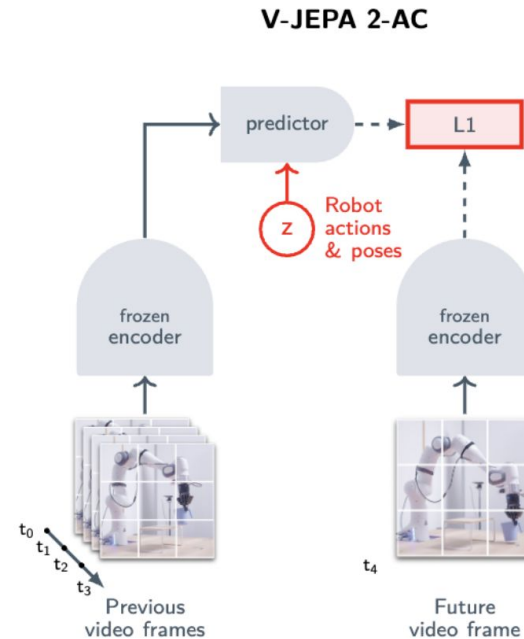
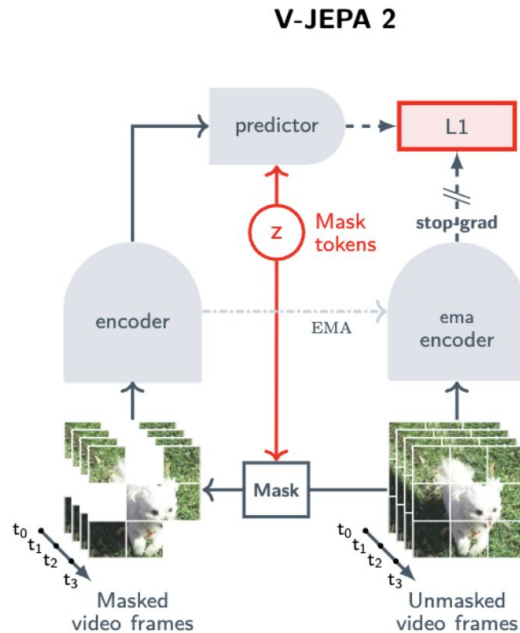
A compact representation that does not capture pixel level detail



What is the latent space

High-Level

A compact representation that does not capture pixel level detail



Background on the Problem

High-Level

Train a goal function on top of a world model that only represents the goal features

01 Planning with latent space world models

A planner scores outcomes of actions against a goal image

02 The goal-image problem

The latent representation of a goal image should encode the true goal, ignoring irrelevant features. For example if a goal image contains a certain robot arm position a world model could encode this into the goal representation

03 Our question

How can we train a goal cost that gives the same object target a similar score across different hand positions in the goal image?

Literature Review

JEPA and DINO are the latent space representation systems we will use.

Visual prediction and planning

- **I-JEPA:** predicts image features rather than pixels
- **DINO-WM:** adds an action-conditioned prediction for planning
- **JEPA-WM:** LeWorldModel study how to plan in latent space

Training JEPA style architectures

- **DINO vs VICReg & LeJEPA:** offer forms of regularization that allow for training without negative sampling

Project Goals

Adapt a frozen DINO-WM to **task-specific planning** without retraining its encoder or dynamics.

01 Define task-relevant success

Match the object's target position while **allowing the robot hand to differ** from the goal image.

02 Learn task-specific goal costs

Learn a simple scoring function that focuses on the object's position, not the robot hand's position.

03 Explore calibration of data allocation

More **object targets** or more **hand configurations** per target?

04 Evaluate planning on unseen targets

Does the learned cost improves **action selection** and closed-loop **task success** on unseen targets?

Approach

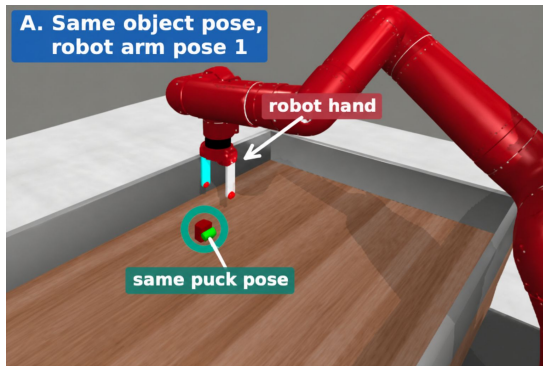
Meta-World Push Simulation Task

- A **robot arm** pushes a small **object** across a table to a **target position**

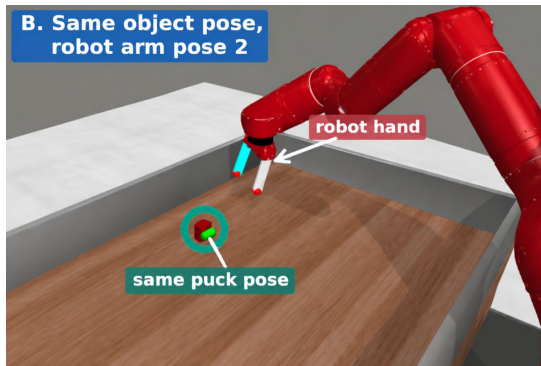
Goal-Cost Learning

- Freeze the encoder and predictor. Train a 384 to 32 linear projection.
- Contrastive loss: bring **same-goal images** closer and **separate different-goal images**

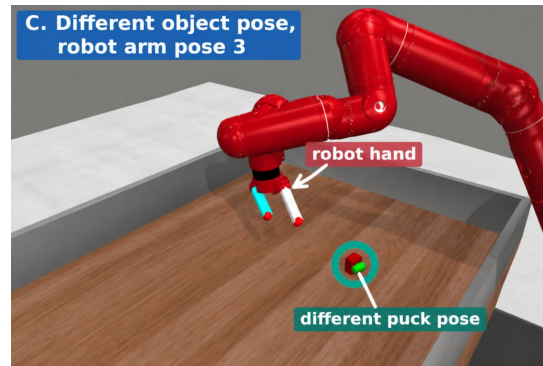
A. Same object pose,
robot arm pose 1



B. Same object pose,
robot arm pose 2



C. Different object pose,
robot arm pose 3



Same-goal Images

Different-goal Images

Approach

Meta-World Push Simulation Task

- A **robot arm** pushes a small **object** across a table to a **target position**

Goal-Cost Learning

- Freeze the encoder and predictor. Train a 384 to 32 linear projection.
- Contrastive loss: bring **same-goal images closer** and **separate different-goal images**

Planning

- Score predicted outcomes using the **average squared distance** between **projected outcome and goal features**.
- Use **Cross-Entropy Method** planning to sample action sequences and refine the lowest-cost candidates.
- Execute a short action segment, observe the environment, and replan.

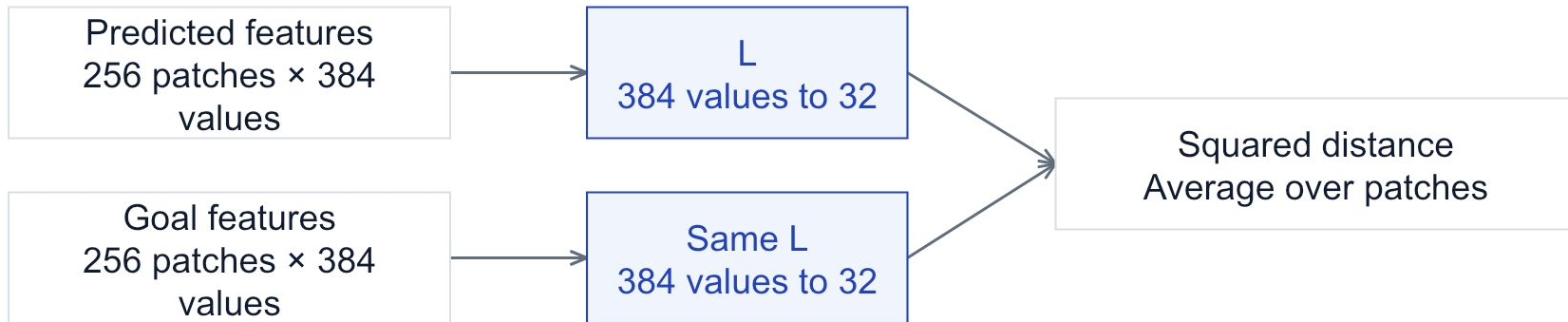
Evaluation

- Measure **placement error**, **success rate**, **sensitivity** to goal-hand appearance, and planning **time**.

Goal cost

$$J_L(A, g) = \frac{1}{N} \sum_{i=1}^N \|L(\hat{z}_i(A) - g_i)\|_2^2$$

A: action sequence g: encoded goal i: image patch N: number of patches



We train the score. The model still predicts with its original features.

Calibration data

256 images = object targets × hand poses per target

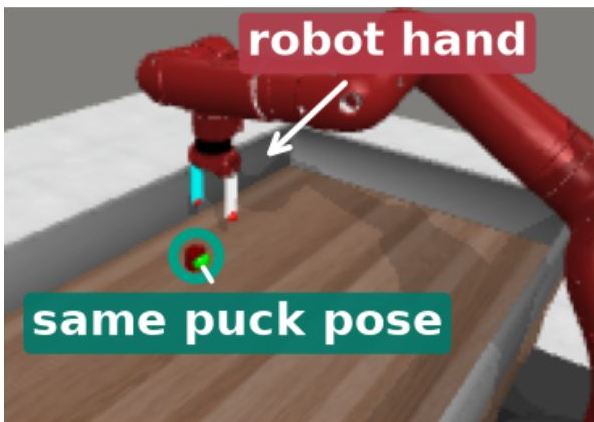
With the same image budget, which examples help more?

Object targets	Hand poses per target	Goal images
128	2	256
64	4	256
32	8	256

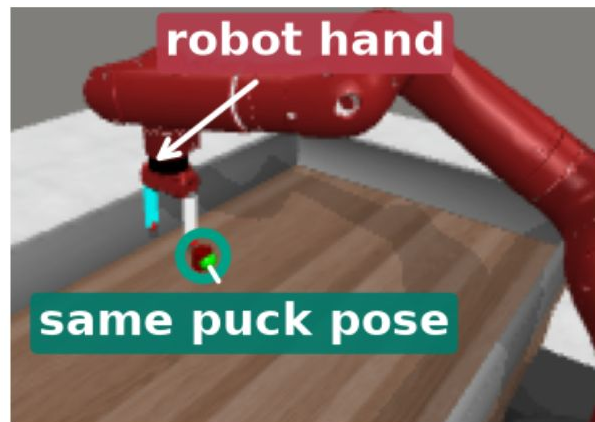
Repeat at 64, 256 and 1,024 images. Keep training updates fixed.

Goal-photo sensitivity

Same puck pose. Different robot hand poses.



Goal photograph 1



Goal photograph 2

Selected action sequence changed in 10 of 16 tasks.

Physical error

51.8%

of the goal-pair difference
energy was removed

8 held-out tasks
Actual endpoint features

Regret = selected error
– best candidate error

Smaller feature differences did not establish better action selection.

Paired error change: +0.24 cm. Recorded 95% interval: -0.25 to +0.81 cm.

Mean extra puck error
relative to the best candidate



Planning evaluation

1

Observed outcomes

Score the same
simulator endpoints.

2

Predicted outcomes

Score the same
model predictions.

3

Executed actions

Run the planner,
observe and replan.

Puck error

Success

Goal-photo sensitivity

Planning time

Compare plain distance, random L and trained L at the same search budget.

Deliverables

Core Objective

Our main deliverable is a reproducible study of whether light task supervision can improve planning with a frozen DINO-WM.

01

Goal-image dataset

Train/val/test splits of grouped meta-world push goal images with controlled object targets and varied robot hand configurations.

02

Trained goal heads

Learned linear metric heads across calibration budgets and target-vs-hand allocations, plus comparison methods.

03

Evaluation Pipeline

Code to run evaluation pipeline. Metrics include regret, 1 / 2 / 5 cm success rate, native task success, and runtime.

04

Final Analysis

A thorough report of how performance changes with calibration data amount/type, alongside measured compute costs and failure analysis.

Why Can't a Coding Agent Just Do This Project?

What coding agents are good at:

- **Implementation:** Loading model checkpoints, implementing training scripts, visualizing outputs via plots.

We need to answer deeper scientific questions:

- **Feature Space Information:** Does the frozen feature space preserve sufficient object information to learn from?
- **Metric Generalization:** Can a small linear metric generalize across hand variations without losing key task signal?
- **Budget Allocation Trade-offs:** Given a fixed data budget, is it better to collect more hand configurations or object targets?
- **Closed-loop Impact:** Does any feature-space improvement survive prediction error to improve closed-loop planning?

Key Takeaway

Once coding and automation are complete, attributing behaviors to their underlying causes remains the true scientific challenge.

Milestones & Timeline

Weeks 1–2

Weeks 3–4

Weeks 5–6

Weeks 7–10

Weeks 11–14

01 Reproduce baseline

- Read I-JEPA, DINO-WM, JEPA-WMs
- Trace model inputs, validate replay

🚩 MILESTONE

Reproduce the goal-photo intervention

02 Train goal heads

- Metric learning, VICReg, LeJEPA
- Fit heads + compression controls

🚩 MILESTONE

Verify head helps predicted outcomes

03 Integrate planning

- Robot-Aware Control, GCR, iCEM
- Matched planning setup

🚩 MILESTONE

Fix thresholds, splits & runtime metrics

04 Run experiments

- Data-allocation comparisons
- Unseen-target tests

🚩 MILESTONE

Verify gains hold when actions are executed

05 Confirm & report

- Confirm the main result
- Add a 2nd task only if justified

🚩 MILESTONE

Audit failures, release artifacts, write report

References

Our Github repo: <https://github.com/AmadeoDeLaVega/JEPA-Planning>

- [1] Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. 2023.
- [2] Zhou et al. DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning. 2024.
- [3] Terver et al. What Drives Success in Physical Planning with Joint-Embedding Predictive World Models? 2025, revised 2026.
- [4] Maes et al. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. 2026.
- [5] Hu et al. Know Thyself: Transferable Visual Control Policies Through Robot-Awareness. ICLR 2022.
- [6] Weinberger and Saul. Distance Metric Learning for Large Margin Nearest Neighbor Classification. JMLR 10, 207–244. 2009.
- [7] Biza et al. On-Robot Reinforcement Learning with Goal-Contrastive Rewards. ICRA 2025.
- [8] Bardes, Ponce and LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. ICLR 2022.
- [9] Balestriero and LeCun. LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics. 2025.
- [10] Yu et al. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. 2019.
- [11] Pinneri et al. Sample-efficient Cross-Entropy Method for Real-time Planning. 2020.
- [12] Agarwal et al. Deep Reinforcement Learning at the Edge of the Statistical Precipice. 2021.

Thank you!